

Experiências Mobile Complementares

Datacrazy

GABRIEL F. HERMENEGILDO

SUMÁRIO

1. Critério de triagem.....	3
2. O que deve continuar só na web.....	4
2.1. Dashboards analíticos.....	4
2.2. Automações e impulsos.....	4
2.3. Agentes de IA, base de conhecimento e servidores MCP.....	4
2.4. Filtros exploratórios completos.....	4
3. O único recurso que vale trazer da web.....	5
3.1. Visibilidade de plano.....	5
3.2. Por que somente leitura.....	5
4. O que só o app consegue entregar.....	6
4.1. Notificação push de mensagem com abertura direta na conversa.....	6
4.2. Lembrete de atividade no horário.....	7
4.3. Agir sobre o contato, em vez de copiar.....	7
4.4. Criar lead a partir do compartilhamento de contato do sistema.....	8
5. Resumo da proposta.....	10
6. Conclusão.....	10

1. Critério de triagem

Antes de listar o que eu traria para o mobile, vale definir o critério usado, porque ele explica tanto as inclusões quanto as exclusões. A pergunta não foi simplesmente "o que a web tem que o app não tem?". A pergunta foi "o que o usuário precisa resolver quando está longe do computador?".

Um CRM é usado em dois contextos diferentes: sentado, analisando e configurando, e em movimento, respondendo e registrando. A web já atende bem o primeiro. O aplicativo deveria ser desenhado para o segundo, e não como uma cópia reduzida do primeiro.

A partir desse critério, a proposta se divide em três grupos: o que deve continuar na web, o único recurso que vale trazer da web e as experiências que só fazem sentido porque existe um celular.

Como o aplicativo atual é React Native CLI puro, cada escolha técnica é apresentada considerando a base bare atual e o cenário com Expo Modules sobre essa mesma base. Quando não existe uma alternativa bare em manutenção ativa, isso fica explícito porque é uma informação que precisa entrar na decisão de infraestrutura.

2. O que deve continuar só na web

Nem tudo que existe no CRM web precisa aparecer no aplicativo. Em alguns casos, tentar colocar a mesma funcionalidade no celular piora a experiência em vez de melhorar.

2.1. Dashboards analíticos

O CRM web possui um conjunto denso de indicadores: total de negócios criados, ganhos e perdidos, valores em aberto, percentual por atendente e, no multiatendimento, atendimentos em aberto e finalizados, origem dos atendimentos, tempo médio de resposta e tempo até o início do atendimento. São gráficos adequados para comparar séries, cruzar períodos e usar um indicador para filtrar outro. Esse tipo de análise pede tela grande e tempo.

No mobile, eu não tentaria reproduzir esses dashboards. Isso transformaria uma ferramenta de análise em uma versão apertada dela. A recomendação é ajustar o que já existe no app, conforme a análise anterior: trocar o gráfico de linha por um indicador compacto de tendência e padronizar quantidade junto com valor em reais.

2.2. Automações e impulsos

São fluxos de configuração com muitas condições encadeadas. Montar isso em uma tela pequena é pior em qualquer cenário, e não é uma tarefa que alguém precise executar enquanto está em movimento.

2.3. Agentes de IA, base de conhecimento e servidores MCP

A lógica é a mesma, com um agravante: são configurações de baixa frequência e alto impacto. Um erro causado por uma tela apertada pode gerar uma consequência grande, então eu manteria esses fluxos na web.

2.4. Filtros exploratórios completos

O filtro de leads da web cobre tags com operação lógica, endereço completo, campos customizados, produtos, listas e origem. No mobile, a proposta já registrada na análise é manter esses recursos disponíveis, mas fixar dois ou três atalhos rápidos na tela. Em movimento, o usuário tende a repetir poucos filtros e usar sempre os mesmos.

3. O único recurso que vale trazer da web

Existe uma informação da web que eu considero importante trazer para o app: a situação do plano. Hoje o site exibe um banner quando o plano gratuito está perto de expirar, enquanto o aplicativo não apresenta essa informação de forma equivalente.

O resultado é que o usuário pode descobrir que o plano expirou pelas consequências. Algumas ações passam a falhar, algumas sem uma mensagem adequada. Mostrar a situação do plano antes disso é uma melhoria simples de transparência e reduz surpresa durante o uso.

3.1. Visibilidade de plano

A proposta é criar uma tela de plano dentro do perfil, em modo somente leitura, com plano atual, status de atividade, data de renovação ou expiração e limites de uso quando aplicável. Junto disso, um aviso dentro do app quando a expiração estiver próxima, seguindo a ideia do banner existente na web.

Tecnicamente, é a parte mais simples desta proposta: consumo de um endpoint existente, uma tela de leitura e um componente de aviso. Não exige biblioteca nova nem configuração nativa.

3.2. Por que somente leitura

A decisão de não colocar contratação e upgrade dentro do aplicativo não é uma limitação técnica. É uma decisão relacionada às políticas de loja.

A diretriz 3.1.1 da Apple determina que conteúdo ou serviço digital pago acessado pelo aplicativo seja comercializado por in-app purchase. Existe uma exceção para serviços

multiplataforma, que permite ao usuário acessar o que contratou em outro lugar, mas ela exige que o mesmo item também esteja disponível como compra dentro do app.

Vale registrar que, desde 2025, a loja americana passou a aceitar botão e link de pagamento externo sem entitlement, por decisão judicial. Isso não se estende às demais lojas. Na loja brasileira continua sendo necessário o entitlement de link externo ou o enquadramento em uma das exceções da diretriz. O tema também segue em disputa quanto à comissão aplicável, então não considero uma base estável para planejamento.

Na prática, colocar contratação no aplicativo abre caminhos com custo e dependências de negócio: implementar in-app purchase nas duas lojas e absorver a comissão, buscar enquadramento como serviço empresarial ou solicitar entitlement de link externo. Como o Datacrazy é um CRM vendido para empresas, o enquadramento empresarial pode ser o caminho mais provável, mas essa decisão envolve jurídico e financeiro, não apenas engenharia.

Exibir o plano é diferente. É somente leitura e entrega a informação que realmente falta. A contratação e a troca de plano continuam na web, onde já funcionam.

4. O que só o app consegue entregar

Os quatro itens abaixo não são versões menores de funcionalidades da web. Eles usam capacidades do dispositivo para resolver tarefas que ficam mais simples quando o usuário está com o celular na mão.

4.1. Notificação push de mensagem com abertura direta na conversa

Hoje, em Preferências do app, notificações aparecem como recurso ainda não ativo. Para mim, essa é a lacuna de maior impacto do produto mobile, porque responder rapidamente a uma nova mensagem é justamente uma das situações em que o aplicativo tem vantagem sobre o navegador.

O recebimento seria feito por `@react-native-firebase/messaging` integrado ao Firebase Cloud Messaging. No iOS, o FCM utiliza APNs por baixo, então a mesma camada atende Android e iOS.

Para exibição, canais e comportamento da notificação, a base bare atual pode usar `react-native-notify-kit`, o fork comunitário do `Notifee`. O `Notifee` foi arquivado pela Invertase em abril de 2026, com última versão em dezembro de 2024, e o fork já foi adaptado para a Nova Arquitetura com TurboModules. Por ser mantido pela comunidade, é uma dependência que precisa ser acompanhada.

Com Expo Modules adotado, `expo-notifications` é a alternativa mais simples e possui manutenção institucional. A contrapartida é que ele não cobre `foreground service` no Android nem alguns estilos avançados de notificação. Para o escopo atual, que é avisar sobre nova mensagem e abrir a conversa correta, as duas opções atendem.

Ao tocar na notificação, o sistema de linking do React Navigation abre a conversa correspondente usando o identificador enviado no payload.

A permissão explícita é obrigatória no Android 13 ou superior e no iOS. O token do dispositivo precisa ser registrado e atualizado no backend por usuário e invalidado no logout. Caso contrário, um atendente que saiu da conta pode continuar recebendo notificações da conta anterior. Também vale testar em aparelhos Android reais, porque otimizações agressivas de bateria de alguns fabricantes podem atrasar a entrega.

4.2. Lembrete de atividade no horário

A agenda de atividades já existe no app, mas fica escondida atrás de um ícone de calendário no topo. A análise já recomendou um contador nesse ícone. No celular, dá para resolver o problema de descoberta de forma mais direta com uma notificação local.

Ao criar ou receber uma atividade com horário definido, o aplicativo agenda uma notificação local para aquele horário. Diferentemente do push, ela não depende de rede nem do servidor. O agendamento fica no próprio aparelho e pode disparar com o app fechado e sem conexão.

A biblioteca segue a mesma escolha do item anterior: `react-native-notify-kit` na base bare e `expo-notifications` no cenário com Expo Modules.

O agendamento precisa ser cancelado e recriado quando a atividade for remarcada e removido quando ela for concluída ou apagada. Por isso, o identificador da notificação agendada deve ser armazenado junto do registro local da atividade.

O iOS também limita a 64 o número de notificações locais pendentes por aplicativo. Em cenários com muitas atividades futuras, a estratégia seria agendar apenas uma janela à frente, como os próximos dias, e reagendar conforme o tempo avança.

O problema de fuso horário já identificado na análise precisa ser corrigido antes deste recurso. Aqui, uma data exibida no dia errado deixa de ser apenas um problema visual e pode fazer o lembrete disparar no horário errado.

Também foi avaliada a gravação da atividade no calendário nativo. Eu deixaria esse recurso fora desta primeira proposta porque as bibliotecas disponíveis para React Native CLI estão sem manutenção ativa, e o ganho é de conveniência, não de necessidade. Se Expo Modules for adotado, expo-calendar pode voltar como incremento posterior. A notificação local é a parte que resolve diretamente o problema do atendente.

4.3. Agir sobre o contato, em vez de copiar

Hoje, tocar no telefone ou no email na listagem copia o valor, sem indicação visual, e na tela de detalhe não há uma ação equivalente. Copiar faz sentido no desktop, onde o passo seguinte pode ser colar em outro programa. No celular, na maioria dos casos, o passo seguinte é realizar a ação.

Na tela de detalhe do lead, eu transformaria telefone e email em ações diretas: ligar, abrir conversa no WhatsApp e enviar email. Isso pode ser feito com o Linking do próprio React Native, usando tel:, mailto: e o link do WhatsApp. Não é necessário adicionar biblioteca.

Copiar continua disponível, mas como opção secundária em um menu. A ação principal fica visível e compatível com o contexto do dispositivo.

O número precisa ser normalizado para o formato internacional antes de montar o link do WhatsApp. Também vale verificar se o aplicativo de destino está instalado antes de oferecer uma ação que poderia terminar sem resposta.

Existe uma diferença importante em relação ao fluxo de abrir chat já existente. O app possui uma ação que depende de uma conexão configurada com WhatsApp, Instagram ou Facebook e registra o atendimento no CRM. Ligar ou abrir o WhatsApp pessoal é um contato direto fora do histórico do sistema. A recomendação é manter abrir chat como ação primária e posicionar ligação e WhatsApp direto como ações secundárias junto ao telefone, deixando claro quando o usuário está saindo do fluxo registrado no CRM.

O comportamento completo de abrir chat com uma conexão ativa não pôde ser validado no ambiente de teste por falta de conta business. Esse ponto deve ser confirmado com o time antes da implementação.

4.4. Criar lead a partir do compartilhamento de contato do sistema

Esse é o recurso que a web não consegue reproduzir da mesma forma. O atendente recebe um contato pelo WhatsApp ou já possui a pessoa na agenda do telefone, usa o compartilhamento do sistema e o Datacrazy aparece como destino. Ao selecionar o app, o formulário de criação de lead é aberto já preenchido.

O escopo é intencionalmente estreito: apenas compartilhamento de contato, não de conversa nem de texto solto. O contato entrega exatamente os campos que o formulário precisa. Compartilhar texto exigiria interpretar conteúdo livre e tentar descobrir o que é nome e o que é telefone, aumentando muito a margem de erro.

No Android, a implementação usa um intent-filter no AndroidManifest.xml aceitando ACTION_SEND para o MIME type de contato. Isso faz o app aparecer na folha de compartilhamento. No iOS, é necessária uma Share Extension, que é um target separado com bundle id e provisioning próprios, executado em processo isolado e comunicado com o app principal por App Group.

A biblioteca apenas automatiza parte dessa configuração. Na base bare, o caminho realista é implementar diretamente o intent-filter e a Share Extension. O react-native-receive-sharing-intent está há cerca de cinco anos sem publicação recente, então eu não adotaria essa dependência para um recurso novo.

Com Expo Modules, expo-share-intent é a alternativa ativa e cobre as duas plataformas por meio de config plugin. Esse é o único dos quatro itens que exige trabalho nativo

relevante, principalmente por causa da Share Extension do iOS. É também um exemplo concreto da estratégia defendida na proposta de migração: resolver uma necessidade pontual com código nativo isolado, sem migrar a aplicação inteira.

O contato compartilhado chega como vCard. O app faz o parse e preenche o formulário com o que estiver disponível. Na prática, nome e telefone tendem a vir preenchidos, enquanto email e tags ficam para o usuário completar.

Um vCard pode trazer mais de um telefone, então o formulário deve permitir escolher qual será usado. O número precisa ser normalizado antes de salvar e vale verificar duplicidade contra os leads existentes, avisando quando o telefone já estiver cadastrado. Esse cuidado é especialmente importante porque o fluxo rápido facilita a criação acidental de registros duplicados.

5. Resumo da proposta

Item	Origem	Biblioteca, base bare	Biblioteca, com Expo Modules	Custo nativo	Observação
Visibilidade de plano	Trazido da web	Nenhuma	Nenhuma	Nenhum	Somente leitura, com status, vencimento e limites quando aplicável.
Push de mensagem com abertura na conversa	Exclusivo do dispositivo	@react-native-firebase/messaging + react-native-notification-kit	@react-native-firebase/messaging ou expo-notifications	Baixo	Notificação deve abrir a conversa correta pelo identificador do payload.
Lembrete de atividade no horário	Exclusivo do dispositivo	react-native-notification-kit	expo-notifications	Baixo	Agendamento local, cancelamento ao remarcar ou concluir e atenção ao limite do iOS.
Ligar, email e WhatsApp direto	Exclusivo do dispositivo	Linking do core	Linking do core	Nenhum	Ações diretas no contato; copiar fica como opção secundária.
Criar lead por compartilhamento de contato	Exclusivo do dispositivo	Implementação direta de intent-filter e Share Extension	expo-share-intent	Médio	Share Extension no iOS e ACTION_SEND no Android; entrada via vCard.

6. Conclusão

A proposta não é transformar o aplicativo em uma versão menor do CRM web. A ideia é concentrar o mobile nas situações em que estar com o celular na mão realmente muda a forma de trabalhar.

Por isso, eu manteria dashboards analíticos, configurações complexas de automação, agentes de IA, base de conhecimento, servidores MCP e filtros exploratórios completos na web. São fluxos que se beneficiam de uma tela maior e de mais tempo para análise ou configuração.

A única informação da web que vale trazer é a visibilidade do plano, em modo somente leitura. É uma implementação simples e resolve um problema real de descoberta antes que o usuário encontre o bloqueio por meio de erros.

Entre as experiências exclusivas do celular, eu priorizaria push de novas mensagens, lembretes locais, ações diretas sobre telefone e email e criação de lead pelo compartilhamento de contato. São recursos pequenos isoladamente, mas todos reduzem uma etapa que no computador exigiria abrir outro sistema, lembrar de consultar uma agenda ou voltar ao CRM.

A arquitetura também reforça a decisão de manter React Native. Três dos quatro itens não exigem código nativo relevante. O compartilhamento de contato exige integração específica de Android e principalmente uma Share Extension no iOS, mas isso pode ficar isolado em um módulo sem transformar a exceção em motivo para reescrever o restante do aplicativo.

O resultado esperado é um app que tenha uma função própria dentro da rotina do atendente: receber uma mensagem e responder rápido, lembrar uma atividade no horário, ligar para um lead, abrir um contato ou transformar um contato recebido no celular em um lead, sem depender de reproduzir toda a experiência de configuração e análise do CRM web.